

Лекция 2. Методика диагностики Linux-сервера

Цель лекции

Понять диагностику Linux-сервера как управляемый процесс, а не набор случайных команд

Отличать симптом, первопричину и сопутствующее событие

Проверять ресурсы, службы, журналы, сеть, DNS и firewall по проверяемым признакам

Безопасно вносить исправления с резервной копией и процедурой rollback

Оформлять диагностический отчет так, чтобы другой администратор мог проверить выводы

Учебные вопросы

1. Симптом, причина и сопутствующее событие; диагностика от общего к частному
2. Baseline исправного сервера
3. CPU, RAM, load average, swar и процессы
4. Диски, файловые системы и свободное место
5. Службы, процессы, порты, загрузка и журналы
6. Сеть, маршрутизация, DNS и firewall
7. Резервная копия, проверка синтаксиса и rollback
8. Документирование и подтверждение исправления

1. Диагностика начинается с точной формулировки проблемы

Симптом - то, что наблюдает пользователь,
мониторинг или администратор

Причина - конкретная неисправность, из-за которой
появился симптом

Сопутствующее событие - событие рядом по времени,
которое может быть связано или случайно совпасть

Пример симптома: сайт web01 не открывается по HTTP

Пример причины: Nginx не запустился из-за ошибки
синтаксиса в конфигурации

Пример сопутствующего события: за час до жалобы
сервер был перезагружен

Событие рядом по времени не доказывает причину

Обновление пакета могло совпасть с заполнением диска, но не вызвать аварию

Перезагрузка могла выявить старую ошибку в /etc/fstab, а не создать ее

Рост нагрузки мог быть следствием отказа службы, а не первопричиной

Администратор проверяет гипотезу командой и фактическим выводом

Хорошая запись: симптом -> гипотеза -> команда проверки -> вывод -> вывод администратора

Диагностика от общего к частному - базовая схема, а не догма

Обычно сначала проверяют доступность сервера, ресурсы, службы, порты, журналы и сеть

Но конкретная ветка диагностики выбирается по симптому и уже найденным фактам

Если сайт работает по IP, но не работает по имени, первой веткой становится DNS

Если сервер отвечает медленно, сначала проверяют CPU, RAM, I/O и приложение

Каждый следующий шаг должен сужать область поиска, а не расширять список догадок

Симптом подсказывает первую ветку проверки

Сервер полностью недоступен: питание, VM, link state, IP, маршрут

Сервер доступен, но служба нет: systemd, порт, журнал, firewall

Работает по IP, но не по имени: DNS, NSS, /etc/resolv.conf, Virtual Host

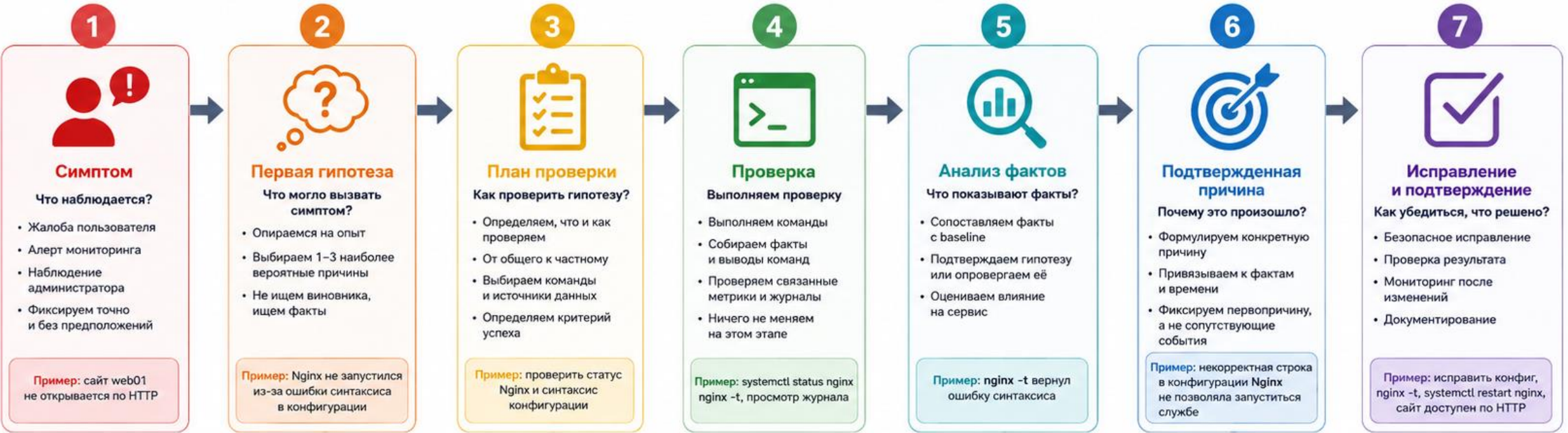
Работает локально, но не извне: bind address, firewall, маршрут, ACL

Служба не запускается: синтаксис, журнал, права, свободное место

Сервер медленный: CPU, RAM, I/O, swap, приложение

Диаграмма: путь от симптома к подтвержденной причине

Структурированный процесс диагностики Linux-сервера от проблемы до подтверждения и исправления.



Сопутствующее событие (не равно причина)

- Событие рядом по времени может быть случайным совпадением
- Перезагрузка, обновление, рост нагрузки, заполнение диска — не всегда причина
- Проверяем гипотезу фактами, а не датой события

11:00

Перезагрузка сервера

11:45

Обновление пакетов

12:30

Рост нагрузки

13:10

Симптом: сайт не открывается

Доказательства важнее предположений

Хорошая запись	Плохая запись
Симптом: сайт не открывается по HTTP	Наверное, проблема в сети
Гипотеза: Nginx не запустился	Скорее всего, упал Nginx
Команда: <code>systemctl status nginx</code>	Проверил что-то в консоли
Вывод: <code>nginx.service failed (code=exited, status=1)</code>	Что-то с ошибкой
Вывод администратора: ошибка синтаксиса в <code>/etc/nginx/nginx.conf:42</code>	Похоже, конфиг битый

Итог

Гипотеза подтверждена

- Исправление
- Проверка результата
- Мониторинг
- Документирование

Гипотеза опровергнута

- Формулируем новую гипотезу
- Сужаем область поиска
- Возвращаемся к шагу 3
- Не расширяем список догадок

2. Baseline показывает нормальное состояние сервера

Baseline - зафиксированное нормальное состояние системы на конкретную дату

Без baseline трудно понять, является ли load average 3 проблемой для данного сервера

Baseline создается после успешной настройки и обновляется после плановых изменений

Он не является вечной нормой: состав служб, нагрузка и маршруты могут измениться

Диагностика сравнивает текущее состояние с известным исправным состоянием

Baseline должен фиксировать ресурсы, сеть и службы

date, hostnamectl, uname -r и uptime фиксируют время, имя, ОС, ядро и время работы

nproc, free -h, df -hT и df -i показывают базовые ресурсы

lsblk -f и findmnt показывают устройства, файловые системы и точки монтирования

ip -br link, ip -br addr и ip route фиксируют состояние сетевых интерфейсов и маршрутов

ss -ltnr и systemctl --failed показывают слушающие порты и неисправные службы

Для критичных пакетов полезно фиксировать версии.

Baseline можно сохранить в текстовый файл

Команда `sh -c` позволяет выполнить набор команд и записать общий вывод в файл

Пример: `sudo sh -c 'date; hostnamectl; uptime; free -h; df -hT; df -i; ip -br addr; ip route; ss -lntup; systemctl --failed' > /root/baseline-$(date +%F).txt`

Файл `baseline` нужен для сравнения при будущей диагностике

В отчете важно указывать дату `baseline`, потому что нормальное состояние меняется

Для учебной работы достаточно текстового файла, но в эксплуатации лучше использовать мониторинг и

Схема: baseline как точка сравнения при диагностике

Baseline — это зафиксированное нормальное состояние сервера, с которым сравнивают текущее состояние при поиске причины проблемы.



Почему baseline полезен



1. Показывает норму именно этого сервера



2. Позволяет отличить симптом от отклонения



3. Ускоряет поиск причины



4. Делает диагностику проверяемой



Пример

Сегодня сайт отвечает медленно. Baseline показывает, что обычно load 0.5–1.0, а сейчас load 6.8 и высокий %wa — значит, внимание нужно сместить на диск или storage, а не только на CPU.

3. Load average показывает не только загрузку CPU

CPU (Central Processing Unit) - процессор, выполняющий инструкции ОС и программ

Load Average в Linux отражает среднее число задач, готовых выполняться на CPU, и задач в непрерываемом ожидании

Непрерываемое ожидание часто связано с дисковым или другим I/O

Высокий load не доказывает, что процессор полностью занят

Команда nproc показывает число доступных логических CPU

CPU-перегрузку и I/O-ожидание различают по нескольким признакам

Команда `uptime` показывает load average за 1, 5 и 15 минут

Команда `top` показывает распределение CPU по режимам

`%id` показывает простой CPU: если он близок к 0%, процессор действительно занят

`%wa` показывает ожидание ввода-вывода: высокий показатель указывает на диск или storage

`%st` показывает Steal Time: гипервизор не дает VM достаточно процессорного времени

Команда `vmstat 1` помогает увидеть `r`, `b`, `us`, `sy`, `id` и `wa` в

top нужно читать не только по списку процессов

%us показывает пользовательскую нагрузку приложений

%sy показывает работу ядра Linux

%wa показывает ожидание диска или другого ввода-вывода

%st особенно важен в VirtualBox, PNetLab и серверной виртуализации

RES показывает реально используемую физическую память процесса

VIRT показывает виртуальное адресное пространство и не равно реальному потреблению RAM

Тяжелый процесс сначала идентифицируют, а не сразу убивают

Команда `ps aux --sort=-%cpu | head` показывает процессы с максимальной CPU-нагрузкой

Команда `ps aux --sort=-%mem | head` показывает процессы с максимальным потреблением памяти

PID (Process ID) - числовой идентификатор процесса

Службы `systemd` обычно останавливают через `systemctl`, например `sudo systemctl stop nginx`

Обычному процессу сначала отправляют `kill -TERM PID` для корректного завершения

`kill -KILL PID` применяют только если процесс не реагирует, потому что `SIGKILL` не дает очистить

Swap не означает аварию сам по себе

RAM (Random Access Memory) - оперативная память сервера

Swap - область диска, куда Linux может перемещать неактивные страницы памяти

Linux может использовать swap до полного исчерпания RAM, освобождая память под файловый кэш

Проблема подтверждается низким available, активным Swap In/Out, задержками и событиями OOM

Команда free -h показывает available, used, buff/cache и swap

OOM Killer указывает на критическую нехватку памяти

OOM (Out Of Memory) - ситуация, когда системе не хватает памяти для продолжения работы процессов

OOM Killer - механизм ядра Linux, который завершает процесс для освобождения памяти

Команда `journalctl -k -b | grep -i -E 'oom|out of memory|killed process'` ищет события OOM в журнале ядра

Если процесс был завершён OOM Killer, нужно искать причину роста памяти, а не просто запускать службу заново

Повторный OOM после restart означает, что причина

Диск проверяется по месту, inode, ошибкам и режиму монтирования

Команда `df -h` показывает заполнение файловых систем

Команда `df -i` показывает использование inode

Команда `lsblk -f` показывает файловые системы и точки монтирования

Команда `findmnt` показывает источник, тип и параметры монтирования

Проблема диска часто выглядит как отказ службы или ошибка записи приложения

Свободное место в `df` не гарантирует, что файловая система доступна на запись

du и find помогают искать рост данных, но создают нагрузку

Команда `sudo du -xhd1 /var` показывает размер каталогов внутри `/var`

Ключ `-x` не позволяет уходить в другие файловые системы

Команда `sudo find /var -xdev -type f -size +100M` ищет крупные файлы

На больших деревьях `du` и `find` выполняют много операций чтения метаданных

На загруженном сервере область поиска ограничивают и выбирают период невысокой нагрузки

Перед удалением файла проверяют назначение через

Удаленный открытый файл может занимать место невидимо для du

Процесс может продолжать удерживать файл журнала после удаления имени из каталога

du уже не видит файл, потому что его имени нет в дереве каталогов

df продолжает показывать занятое место, пока процесс не закроет файл

Команда `sudo lsof +L1` показывает открытые удаленные файлы

Дополнительная проверка: `sudo lsof | grep deleted`

Место освобождается после закрытия файла

процессом или перезапуска соответствующей службы

Файловая система может стать Read-Only при ошибках накопителя

Read-Only означает, что файловая система смонтирована только для чтения

Служба может падать с ошибкой записи, хотя `df -h` показывает свободное место

Команда `findmnt -no SOURCE,FSTYPE,OPTIONS /` показывает параметры монтирования корневой ФС

Если в `options` присутствует `ro`, запись в файловую систему запрещена

Команда `journalctl -k -b -p warning` помогает найти предупреждения ядра

Причину нужно искать в ошибках диска. RAID.

Kernel Journal точнее, чем случайный хвост dmesg

Команда `dmesg -T | tail` полезна для быстрого просмотра последних сообщений ядра

Последние строки не всегда относятся к моменту аварии

Команда `sudo journalctl -k -b` показывает журнал ядра текущей загрузки

Команда `sudo journalctl -k -b -p warning` показывает предупреждения и ошибки ядра

Фильтр `sudo journalctl -k -b | grep -i -E`

`'error|fail|i/o|ext4|xfs|nvme'` помогает искать дисковые признаки

5. active в systemd не доказывает исправность приложения

systemd - система инициализации и управления службами в современных Linux-дистрибутивах

Команда `systemctl status nginx` показывает состояние процесса и последние сообщения

Служба может быть active (running), но приложение возвращать HTTP 500

Nginx может слушать только 127.0.0.1, а клиент из сети не сможет подключиться

Сертификат, upstream или Virtual Host могут быть неисправны при активной службе

Проверка должна идти на трех уровнях: служба -> порт

Служба, порт и приложение проверяются разными командами

Команда `systemctl is-active nginx` проверяет, считает ли `systemd` службу активной

Команда `sudo ss -ltnp | grep :80` проверяет, слушается ли TCP-порт 80

Команда `curl -v http://127.0.0.1/` проверяет локальный HTTP-ответ приложения

Команда `curl -v http://192.168.10.20/` проверяет доступ по адресу сервера

Если один уровень успешен, а следующий нет, область поиска сужается

Диагностика приложения не заканчивается на слове

Bind Address объясняет, почему локально работает, а из сети нет

Bind Address - адрес, на котором процесс принимает сетевые подключения

127.0.0.1:80 означает доступ только с самого сервера

0.0.0.0:80 означает прослушивание на всех IPv4-интерфейсах

:::80 означает прослушивание IPv6, а поведение IPv4 зависит от настроек системы

192.168.10.20:80 означает прием только на конкретном адресе сервера

Если локальный curl работает, а клиент нет, нужно проверить bind address, firewall и маршрут

Журналы нужно читать вокруг времени симптома

Команда `journalctl -u nginx -b` показывает журнал службы за текущую загрузку

Пример по времени: `sudo journalctl -u nginx --since '2026-07-28 10:00:00' --until '2026-07-28 10:15:00'`

Быстрый вариант: `sudo journalctl -u nginx --since '-30 min'`

Команда `journalctl -u nginx -b -1` показывает предыдущую загрузку только при наличии сохраненного журнала

Если `journald` хранит журнал только в памяти, события предыдущей загрузки после перезагрузки исчезнут

Синтаксис проверяется до применения конфигурации

Для Nginx команда `sudo nginx -t` проверяет конфигурацию без перезапуска

Для SSH команда `sudo sshd -t` проверяет синтаксис `sshd_config`

`reload` перечитывает конфигурацию без полного завершения процесса

`restart` полностью останавливает и запускает службу заново

Не каждая служба поддерживает `reload`

Если служба уже остановлена, обычно нужен `start` или `restart`, а не `reload`

Изменение SSH требует отдельной страховки

SSH (Secure Shell) - защищенный удаленный доступ к серверу по командной строке

Перед применением `sshd_config` выполняют `sudo sshd -t`

Текущую SSH-сессию нельзя закрывать до проверки новой конфигурации

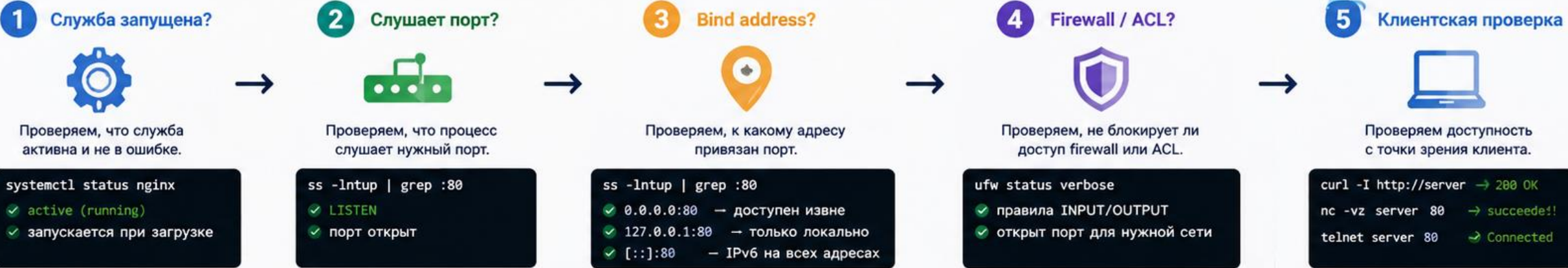
Нужно открыть вторую тестовую сессию и убедиться, что вход работает

В учебной VM полезно иметь консоль гипервизора или snapshot перед изменением SSH

Ошибочная настройка SSH может заблокировать

Схема: служба, порт, bind address и клиентская проверка

Путь проверки: служба запущена → слушает порт → привязана к адресу → доступна клиенту



Как интерпретировать bind address			
Bind address	Пример в ss	Значение	Клиенты
0.0.0.0	0.0.0.0:80	Слушает на всех IPv4-адресах сервера	Доступен локально и извне (IPv4)
127.0.0.1	127.0.0.1:80	Слушает только на локальном интерфейсе (loopback)	Доступен только локально
[::]	:::80	Слушает на всех IPv6-адресах	Доступен локально и извне (IPv6)
192.168.1.10	192.168.1.10:80	Слушает только на конкретном интерфейсе	Доступен только из сети этого интерфейса

Пример: Nginx на порту 80		
	1. Служба	<pre>systemctl status nginx</pre> active (running) ✓
	2. Порт	<pre>ss -ltnup grep :80</pre> LISTEN 0 128 0.0.0.0:80 users:("nginx",pid=1234,fd=6) ✓
	3. Bind	Привязан к 0.0.0.0:80 → доступен извне (IPv4)
	4. Firewall	<pre>ufw status verbose</pre> 80/tcp ALLOW IN Anywhere ✓
	5. Клиент	<pre>curl -I http://server</pre> HTTP/1.1 200 OK ✓

Если не доступно — куда смотреть		
	Служба не запущена	→ <pre>systemctl status <service></pre> <pre>journalctl -u <service> -xe</pre>
	Порт не слушается	→ конфигурация службы ошибка запуска
	Bind только на 127.0.0.1 или конкретном IP	→ изменить конфиг (listen/bind) перезапустить службу
	Firewall блокирует	→ открыть порт / настроить правила
	Клиент не может подключиться	→ маршрут, DNS, ACL, Cloud Security Group и т.д.

Полезные команды

Службы

```
systemctl status <service>  
systemctl is-enabled <service>  
journalctl -u <service> -n 50 --no-pager
```

Порты и процессы

```
ss -ltnup  
ss -ltnup | grep :80  
lsof -iTCP -sTCP:LISTEN -P -n
```

Сеть

```
ip -br a  
ip route  
ip addr show
```

Firewall (UFW / iptables)

```
ufw status verbose  
iptables -S  
iptables -L -n -v
```

Клиентские проверки

```
curl -I http://server  
nc -vz server 80  
telnet server 80
```

Итог

Услуга доступна клиенту, когда:
служба запущена, порт слушается,
привязка позволяет доступ, firewall
не блокирует и клиент получает ответ.

6. Сетевую диагностику начинают с link state

Link State - состояние сетевого интерфейса на канальном уровне

Команда `ip -br link` показывает, существует ли интерфейс и находится ли он в состоянии UP

Команда `ip -br addr` показывает IP-адреса и маски интерфейсов

Команда `ip route` показывает локальную таблицу маршрутизации

Команда `ip neigh` показывает ARP/Neighbor-таблицу соседних узлов

Сначала проверяется локальный стек сервера, а уже

Маршрут проверяется на управляемых целях лаборатории

В изолированном стенде Интернет может отсутствовать, поэтому 8.8.8.8 не должен быть единственной целью

Сначала проверяют собственный IP и интерфейс

Затем проверяют default gateway, например ping -с 4
192.168.10.1

После этого проверяют соседний сервер или сервер в другой подсети

Команда `ip route get 192.168.30.10` показывает выбранный маршрут, source IP и интерфейс

`ip route get` не доказывает прохождение пакета до

DNS проверяется отдельно от IP-доступности

DNS (Domain Name System) - система преобразования доменных имен в IP-адреса

`getent hosts web01.company.test` проверяет системный механизм разрешения имени через NSS

`getent ahosts web01.company.test` показывает адреса разных семейств, если они доступны

`dig @192.168.30.53 web01.company.test` выполняет прямой запрос к конкретному DNS-серверу

`resolvectl status` доступен только на системах с `systemd-resolved`

`/etc/resolv.conf` может быть символической ссылкой и

Firewall сначала определяют по механизму управления

Firewall - механизм фильтрации сетевого трафика по правилам

На Ubuntu часто используют UFW поверх nftables: `sudo ufw status verbose`

На RHEL-подобных системах часто используют firewalld: `sudo firewall-cmd --state` и `sudo firewall-cmd --list-all`

nftables напрямую проверяют командой `sudo nft list ruleset`

iptables может быть compatibility layer поверх nftables, поэтому вывод `iptables -S` не всегда показывает всю

Локальная проверка не заменяет проверку с клиента

Если `curl http://127.0.0.1/` не работает, проблема обычно в службе или приложении

Если `localhost` работает, а `curl http://192.168.10.20/` нет, вероятен `bind address`

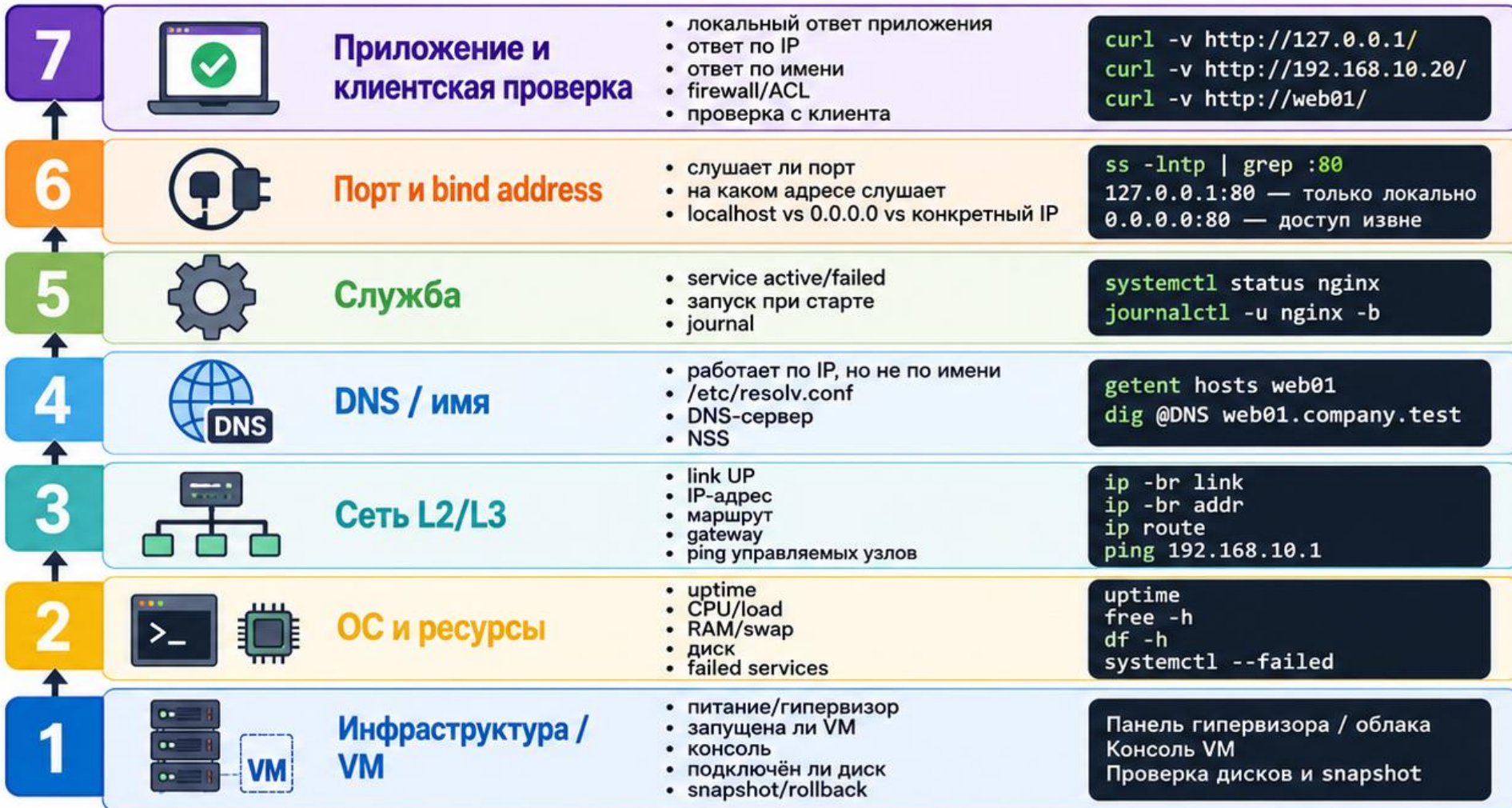
Если сервер локально отвечает по своему IP, а клиент нет, проверяют `firewall`, маршрут и `ACL`

Если работает по IP, но не по имени, проверяют `DNS` или `Virtual Host`

Проверка должна повторять исходный пользовательский сценарий, иначе исправление не доказано

Диаграмма: диагностика сервера по уровням

Проверка идёт по уровням: от инфраструктуры и ОС к сети, службе, порту, приложению и клиенту.



Как читать уровни



Если нижний уровень неисправен — выше проверять рано



Каждый шаг сужает область поиска



Событие рядом по времени — не доказательство причины



Исправление подтверждаем повторением сценария клиента

Логика диагностики



Причина подтверждена, когда найдено отклонение и проверка после исправления успешна.

7. Перед изменением нужна точка возврата

Rollback - возврат к предыдущему рабочему состоянию после неудачного изменения

Простая копия файла лучше, чем отсутствие копии, но для конфигурации важны атрибуты и время

Пример: `sudo cp -a /etc/nginx/nginx.conf
/etc/nginx/nginx.conf.bak-$(date +%F-%H%M%S)`

Nginx часто хранит конфигурацию не только в `nginx.conf`, но и в `conf.d`, `sites-available` и `sites-enabled`

Если меняется несколько файлов, копируют все изменяемые файлы или весь конфигурационный каталог

Git, snapshot VM и backup решают разные задачи

Копия файла дает быстрый локальный откат одной конфигурации

Git хранит историю текстовых конфигураций, но репозиторий нужно защищать от утечки секретов

Snapshot VM сохраняет состояние всей виртуальной машины и удобен для учебного Break/Fix

Snapshot VM зависит от того же хранилища и не является долгосрочным backup

Backup - отдельная восстанавливаемая копия вне исходной системы

Для продакшена rollback должен опираться не на один

Безопасное изменение включает проверку и план отката

Сначала создать копию: `sudo cp -a /etc/nginx/sites-enabled/app.conf /etc/nginx/sites-enabled/app.conf.bak-$(date +%F-%H%M)`

Затем отредактировать файл через `sudoedit /etc/nginx/sites-enabled/app.conf`

Перед применением выполнить `sudo nginx -t`

Если тест успешен, применить `sudo systemctl reload nginx`

Проверить приложение командой `curl -v http://127.0.0.1/`

Если проверка неуспешна, вернуть сохраненный файл.

8. Отчет должен доказывать причину, а не перечислять команды

Формат отчета: симптом -> гипотеза -> команда
проверки -> фактический вывод -> причина ->
исправление -> результат

Дополнительно фиксируют дату, hostname, IP сервера
и кто сообщил о проблеме

Нужно указать ожидаемое и фактическое поведение

В отчет включают только ключевые строки вывода,
подтверждающие вывод администратора

Отдельно фиксируют, где находится backup и как
выполнить rollback

В конце указывают профилактическую меру, если она

Нормальные и аварийные выводы команд нужно уметь читать

Служба работает: Active: active (running)

Служба неисправна: Active: failed (Result: exit-code)

Порт слушается извне: LISTEN 0 511 0.0.0.0:80

Порт слушается только локально: LISTEN 0 511
127.0.0.1:80

Есть маршрут по умолчанию: default via 192.168.10.1
dev enp0s3

Проблема DNS может выглядеть как Temporary failure
in name resolution

Пример диагностического отчета по web01

Симптом: HTTP-сервис web01 недоступен с client01

Гипотеза: Nginx не слушает TCP 80 из-за ошибки конфигурации

Проверка: `systemctl status nginx; sudo ss -lntp | grep :80; sudo nginx -t`

Причина: ошибка синтаксиса в `/etc/nginx/sites-enabled/app.conf`

Исправление: восстановлена закрывающая фигурная скобка, выполнены `nginx -t` и `systemctl reload nginx`

Проверка результата: `curl http://127.0.0.1` и `curl http://web01` с client01 возвращают HTTP 200

Break/Fix-лабораторная требует доказательства каждого исправления

Web-сценарий: синтаксическая ошибка Nginx или bind только на 127.0.0.1

Дисковый сценарий: заполнен /var, закончились inode или есть открытый удаленный файл

Сетевой сценарий: удален default route, неправильный DNS или закрыт TCP 80

Ресурсный сценарий: процесс создает давление на CPU или RAM

Для каждого сценария оформляется симптом, гипотеза, проверка, причина, backup, исправление, проверка и rollback

Когда нужно остановиться и эскалировать проблему

Если исправление может привести к потере данных, нужна пауза и резервная копия

Если проблема затрагивает несколько серверов, нужно проверить общую зависимость: DNS, сеть, storage или authentication

Если команда непонятна, ее нельзя запускать от root на рабочем сервере

Если нет прав или доступа к компоненту, это фиксируется в отчете

Эскалация - нормальная часть профессиональной эксплуатации. когда риск выше полномочий или

Итоги лекции

Диагностика начинается с симптома, но маршрут проверки выбирается по фактам

Baseline помогает отличить норму конкретного сервера от отклонения

CPU, RAM, диск, службы, порты, сеть, DNS и firewall проверяются отдельными признаками

active в systemd не заменяет проверку порта, bind address и ответа приложения

Перед изменением конфигурации нужен backup, синтаксическая проверка и план rollback

Хороший отчет содержит фактические доказательства

Контрольные вопросы

Чем симптом отличается от причины и сопутствующего события?

Почему диагностика от общего к частному не должна быть жесткой догмой?

Какие данные должен содержать baseline исправного Linux-сервера?

Почему высокий load average не всегда означает перегрузку CPU?

Какие признаки подтверждают реальную проблему памяти, а не просто использование swp?

Почему df и du могут показывать разные значения